CS673 Software Engineering
Team 3 - SafeDrop
Project Proposal and Planning

Team Member	Role(s)	Signature	Date
Alexa Stein	Team Lead	*Alexa Stein*	10/12/2026
Amber Rastella	Security	*Amber Rastella*	10/12/2026
Alex Picard	Configuration	*Alex Picard*	10/12/2026
Mateus Silva	Requirements	*Mateus Silva*	10/12/2026
Orelmis Toribio	QA	*Orelmis Toribio*	10/12/2026

Revision history

Version	**Author**	**Date**	**Change**
1.0	**Alex Picard**	**9/20**	**Added to verified column and added references**
1.1	Amber Rastella	9/21/2026	Iteration 1 revision (SCRUM-131). Corrected the authentication decision to self-managed auth; revised the Iteration 1 effort estimate against the kickoff breakdown; added the dev/prod parity, key-person and external-hostname risks and the SCRUM-114 and SCRUM-125 security items to the risk register.

1.2	Alex Picard	9/30/2026	Added to “Verified By” Section
1.3	Alex Picard	10/05/2026	Edited NFR-5
1.4	Mateus Silva	10/10/2026	Iteration 2 timeline update (SCRUM-221). §4c now records AI-assisted search brought forward from Iteration 3 and semantic search (SCRUM-147) moved to Iteration 3, each with its reason, and Iteration 2 actual hours (90 against 65 estimated). Added the matching §7 AI usage row.

1. Overview
2. Related Work
3. Proposed High level Requirements
4. Management Plan
    a. Objectives and Priorities
    b. Risk Management (need to be updated constantly)
    c. Timeline (this section should be filled in iteration 0 and updated at the end of each later iteration)
5. Configuration Management Plan
    a. Tools
    c. CI/CD Plan
6. Quality Assurance Plan
    a. Metrics
    c. Code Review Process
    d. Testing
    e. Defect Management
7. AI usage Log
8. Project Links
9. References
10. Glossary

1. Overview

We are building an organizational inventory management/checkout service. The motivation behind this project is to provide organizations (whether it be corporations, educational institutions etc.) with a centralized, easy to navigate interface that can be used to facilitate authorized checkouts of the organization's physical assets/equipment to its members while maintaining detailed audit logs about every interaction involved in that process (requests, approvals, checkouts, returns – a full history for every individual asset). For example, a university may want to supply their students with laptops to borrow over the course of a semester, and this would allow them to keep track of those rather expensive assets. This will take the form of a full stack web application that will allow organizations to create their own inventory portals, with an interface allowing organization members to request and checkout equipment as well as an administrator dashboard for managing the inventory itself and granularly configuring access restrictions/approval workflows. The application will be built using the MERN stack (Mongo DB, Express, React, Node).

2. Related Work

a. **Freshervice** is a ticketing system that offers asset management to an IT Department that helps track what users have what assets/software.
    - As tickets are coming in, it can associate assets with that user and automatically service it through remote options
    - Freshservice is similar to our software in that it offers tracking of equipment/asset management.
    - Freshservice is different from our software in that it mainly focuses on offering ticketing services, and their asset management caters to this purpose rather than focusing on tracking as our main core to our software.
b. **Libby** is an app that you can use your library card to rent e-books for free to your kindle
    - Libby is similar to our software by managing and tracking requests, approvals, checkouts, returns and keeping history of the assets
    - Libby is different from our software because the approvals process will behave differently. On libby the approvals are based on if the library has enough e-book licenses available at the time while our software (not in P1) will have more intricate algorithms behind the health of the asset and the borrower.
c. **Point of Rental** is an app that specializes in providing software to equipment rental companies, and only having the customers pay for what they use.
    - Point of Rental is similar, as it keeps track of inventory, tracks the downtime of products, and provides a checkout service to the end user.
    - Point of Rental is different as it is specialized in the specific area of equipment rentals, when our software could allow for any single item to be uploaded by the company; it would just need some configuration by the company/user of the product. Additionally, this software would be cheaper, as quick research has shown that Point of Rental is in the range of $500+ per month.

3. Proposed High level Requirements

a. Functional Requirements
    - Essential Features (the core features that you definitely need to finish):
        - Portal user UI) - 15 hrs
            a. As an end user Admin, I want to create a portal so that I can manage my organization's assets + equipment
        - Check out/Return Items - 15 hrs
            a. As an end user, I want to check out/return items
        - Audit log of who returned or checked out items - 8 hrs
            a. As an Admin end user, I want to log and view which users requested/returned items, as well as which Admins added/removed/updated item information and statuses
        - Login functionality - 12hrs

a. As an end user, I want to log into my dashboard to check my requests, return items, and make new requests
b. admin dashboard (including management of item assets) - 5 hrs
c. Estimate revision (Iteration 1, SCRUM-131). The five essential features above were estimated at 55 person-hours in total, and Iteration 1 at 70. Both figures were low. The Iteration 1 kickoff broke the audit-log epic down to roughly 32 hours against its 8-hour estimate (+24), and scoping the frontend added roughly 25 hours that no feature estimate had accounted for. On that breakdown the essential features are about 104 person-hours and Iteration 1 about 119 — a 70% underestimate. The original per-feature figures are left above as written so the size of the error stays visible; the revised totals are what Iteration 2 planning should be sized against. Actual logged hours are recorded per member in CS673_ProgressReport_team3.

- Desirable Features:
  - Admin options to set up approval flows for checking out items (could be for all items or only specific ones)
  - privileged users (customer tiers) or groups of users
  - equipment or groups of equipment that require elevated permissions
  - AI assisted search (product recommendations)
- Optional Features::
  - AI capabilities to look over a particular user's history to determine their reliability (ie. if they ever return anything late or with damages, that would be logged in their history, and the AI could use that information to make determination)
  - AI projection of the future depreciation and value of the assets owned by the organization
  - Convert JS files to Typescript to increase security and lower bugs (utilize AI)
  - Requests for additional equipment – approval flow for creating purchase order

b. Nonfunctional Requirements
- nonfunctional-requirements

4. Management Plan

a. Objectives and Priorities

For our project, the following objectives are listed in priority order:

1. Complete all essential features and requirements

   a. A working prototype is needed as the base before all the enhancements can be added
2. Test the application and code to reduce bugs and deliver a reliable product
   a. This ensures that we maintain a high code quality and have thorough and comprehensive testing
3. Deploy the code effectively, successfully and safely
4. Ensure a positive user experience for our customers
5. Communicate and collaborate effectively within our development team

b. Risk Management (need to be updated constantly)

Risk Management Sheet Link: CS673_SPPP_RiskManagement

c. Timeline

Iteration	Functional Requirements(Essential/ Desirable/Optional)	Tasks (Cross requirements tasks)	Estimated/real total person hours
1	Essential	Set up databases, backend infrastructure, system design, and basic UI	70 estimated / 119 revised (see note under Essential Features)
2	Desirable, plus AI-assisted search brought forward from Iteration 3	Customer focused unique enhancements: configurable approvals, user groups, restricted equipment, two-sided custody, AI-assisted search and AI-recommended alternatives	65 estimated / 60 actual
3	Optional	Integrate AI and additional security measures	40

Iteration 2 took 60 person-hours against a 65-hour estimate, and its scope changed twice: AI-assisted search came forward from Iteration 3, and semantic search (SCRUM-147) moved out to Iteration 3.

Iteration 2 scope change details:

- AI-assisted search moved forward from Iteration 3 to Iteration 2 (SCRUM-103, 145, 146, 200, 201; reused by SCRUM-151). Reason: it is a Desirable feature in §3a, and SCRUM-103 was built test-first as the team's Lab 3 work, so it shipped with the other Iteration 2 enhancements.

- Semantic search (SCRUM-147) moved out of Iteration 2 to Iteration 3, with search telemetry (SCRUM-206). Reason: SCRUM-147's code was not in main when Iteration 2 closed, so it was reopened and re-planned in the Iteration 3 sprint, starting with a vector-store spike; SCRUM-206 had not started.

5. Configuration Management Plan

a. Tools

- Frontend: React, JS
- Backend: ExpressJS, Node.js
- Authentication: self-managed email + password (bcrypt cost 12, JSON Web Tokens, rotating refresh tokens). Decided and implemented in Iteration 1 — see SDD §6.2.
- DBMS: MongoDB Atlas
- Version Control: Git/Github
- Container: Docker, Docker Compose
- AI: Microsoft Foundry-> Will test between different models.
- Project Management: Jira
- CI/CD Tools: GitHub Actions
- SAST: GitHub CodeQL
- DAST: OWASP ZAP
- Hosting: Render for backend, Vercel for frontend
- If time allows:
- (Clerk and Auth0 were evaluated in Iteration 0 and not adopted. SafeDrop implements its own authentication; Clerk was removed from the codebase. This keeps the tenant and role claims, cookie configuration and session lifetime under the team's control, at the cost of owning security-critical code — mitigated by using maintained crypto libraries and requiring tests on every authentication path.)
    - AWS tools: S3, Budget/Billing Alarms, CloudFront, Lambda, SSM Parameter Store, IAM, CloudWatch, API Gateway.

b. Code Commit Guideline and Git Branching Strategy

- GitHub Flow Style:
  For this project, we will be following the GitHub Flow strategy closely. The goal is to keep the main branch clean while working on features, bugs, etc. on a designated branch; once the branch's goal is complete, merge it back

into main. The main goal of this strategy is to keep the main branch clean, and it supports an agile style in a small-group setting. The pull requests will be submitted when a branch is ready to be merged and will be reviewed by a secondary member of the team before merging to main in order to get a second look at the code. All pull requests should be either merged or request changes prior to the end of each iteration.

c. CI/CD Plan

- The CI/CD plan is to use GitHub Actions. The primary goal is to have an automated test suite run on every pull request and push. Once the main branch is green, this will trigger a deployment for staging. At the end of each iteration, a tagged release will then promote the code to the production server. The initial plan is to start with a Render Backend with a Vercel frontend to hit the ground running, and if enough time permits, then an AWS production environment will replace it.

6. Quality Assurance Plan

a. Metrics

Metric Name	Description
Line Coverage	Percentage of LOC that are covered by tests (ideally >= 80%). Measures the comprehensiveness of our tests.
Average LOC/file	Average LOC per file (excluding configuration files, comments, empty lines). Estimates the modularity of our application and the complexity of our modules.
Average LOC/function	Average LOC per function (excluding comments and empty lines). Estimates the complexity of our functions.
Test case pass rate	Percentage of current tests that pass. Measures how well the application's behavior meets our expectations.
Average Fanout/file	Average number of imported dependencies per source file (excluding third party libraries). Measures how tightly coupled it is to other modules in our project.

b. Coding Standard

For the ease of testing and maintenance, care should be given to the separation of concerns and modularity of the code. For example, rather than executing a raw database query directly within an express route handler, the handler should call a

helper/utility function to perform that operation (i.e `getUser(userId)`). This makes it easier to test `getUser` in isolation, and to mock that functionality in other tests. A similar approach should be used for API calls from the React frontend to the Express backend.

Use clearly defined, easily navigable folder structures. For example (rough idea):

- For REACT: src/components, src/pages, src/hooks, src/services, src/context, src/utils, src/App.jsx, src/main.jsx, public/, tests/, etc.
- For Express: src/routes, src/controllers, src/services, src/models, src/middleware, src/repositories, src/app.js, src/server.js, src/utils, tests/ etc.

Otherwise, use standard Javascript naming, spacing, formatting conventions.
Ex: https://www.w3schools.com/js/js_conventions.asp

c. Code Review Process

We will have team members submit their code by making a pull request to merge their feature branch with the main branch. The code will have to pass the existing tests for the merge to go through, and the pull request will need to be approved by another member of the team. The reviewer(s) will need to check that the code is logically correct, that the unit test coverage for the new feature is sufficiently comprehensive, and that there aren't any other obvious issues with the code (missing error handling or edge case, poor modularity etc.) If the reviewer does find an issue, they should leave a review comment with feedback which will need to be addressed before the pull-request is reopened.

d. Testing

Unit Testing

- Every substantial new feature should be accompanied by unit tests that define the expected behavior of that feature. Team members should create these unit tests before attempting to merge their changes to the main branch. These tests should mock their dependencies (we can also set up a test database so the queries themselves are tested as well).
- React Front end tools: **Vitest + React Testing Library** – allows testing of React components without starting the app
- Express Backend tools: **Vitest + Supertest** – allows testing of express routes without starting an HTTP server
- Note: Vitest will be used to generate line coverage reports by the QA leader at the end of each iteration.

E2E Testing (Nice to have)

- Before deployment, we should also test that these features work end to end (no mocking).
- E2E tools: **Playwright**

These tests should should cover happy paths as well as unhappy paths (express endpoints should return the appropriate error codes/messages, and the React UI should give the user the appropriate visual feedback, including error states)

e. Defect Management

We will track defects with Jira. These defects will include any behavior in the code that deviates from expectations (unexpectedly failing tests, a UI bug, an error message, performance issues, security issues, etc). The person who discovers the defect should create an issue for it with a detailed description of the issue, and the reproduction steps. Add the bug into the backlog – responsibility assigned in weekly meetings.

7. AI usage Log

(This table summarizes AI contribution in this document)

Section	Who	AI contribution (%)	Description	Verified by
5a	Alex	5%	Comparing auth services Auth0 and clerk	https://dev.to/mechcloud_academy/clerk-vs-auth0-choosing-the-right-authentication-solution-3cfa
2	Alex	5%	Get a cost estimate for Point of Rental	Checking similar product's pricing. Cannot find pricing without asking for demo.
3b,4b	Amber	50%	Non-functional requirements drafted by AI	Alex Picard
4b, 5a, 3a	Amber	60%	Claude Code reconciled the SPPP	Alex Picard

Section	Who	AI contribution (%)	Description	Verified by
(estimates)			against the merged Iteration 1 code: corrected the authentication decision, revised the Iteration 1 estimate from the kickoff breakdown, and drafted the new risk register rows. Every claim was checked against the repository.	
4c	Mateus	70%	Claude drafted the Iteration 2 timeline update (SCRUM-221) from the Iteration 2 progress report, presentation and Jira notes: both scope changes with reasons, and Iteration 2 actual hours totalled from the weekly reports.	Alex looked over the sections and looked correct

Student	**Log Link**
Amber Rastella	AI_Usage_Log_Amber

8. Project Links

Github link: https://github.com/BUMETCS673/CS673OLF26P3
Project management tool link (e.g. Jira):
https://bu-team-x3gl1rkg.atlassian.net/jira/software/projects/SCRUM/boards/1

9. References

https://www.point-of-rental.com
https://docs.google.com/document/d/1543BPMMIEFC077MNHdoDC9CiDL7eHyUiLw91R4d_gls/edit?usp=drive_link
CS673_SPPP_RiskManagement

https://www.w3schools.com/js/js_conventions.asp

10. Glossary

- LOC - line of coverage
- SR - Security requirement
- NFR - Non-functional requirement